Lập trình sử dụng API

Mở đầu – Cửa sổ và thông điệp



Nguyễn Thu Hường

Khoa CNTT – Đại học GTVT

Nội dung

- Mục tiêu môn học
- Giới thiệu tài liệu tham khảo
- Các khái niệm cơ bản về API
- Viết chương trình Windows đầu tiên
- Cửa sổ và thông điệp

Mục tiêu môn học

- Miêu tả cách thức viết các chương trình chạy trong các hệ điều hành của Microsoft đặc biệt là hệ điều hành windows
- Giới thiệu các khái niệm về Web service.
- Cách thức xây dựng API dựa trên chuẩn Rest.
- Làm quen với một số API: facebook API, Map box API…

Ngôn ngữ và môi trường lập trình

- Ngôn ngữ lập trình: C++, C#, Python…
- Một số ngôn ngữ khác: javaScript, HTML…
- Môi trường lập trình gói sản phẩm Microsoft Visual Studio phiên bản 2015 trở lên.

Đánh giá học phần

- Điểm đánh giá quá trình: 40%
  - Chuyên cần: 5%
  - Bài tập lớn: 20%
    - BTL: mỗi nhóm gồm 3-5 SV xây dựng một ứng dụng Win API hoặc sử dụng các Web API.
    - Báo cáo vào 2 tuần cuối của môn học
    - Sinh viên không tham gia BTL được 0 quá trình.
  - Kiểm tra bài tập (thực hành trên máy tính): 15%
- Điểm thi kết thúc học phần: 60%
  - Hình thức: thi trên máy (thời gian 60 phút)

Tài liệu tham khảo



- Các slides dùng để giảng dạy trên lớp của giảng viên.
- Bản dịch của giáo viên Lưu Xuân Duẩn từ tài liệu "Programming Windows" xuất bản lần thứ 5 của tác giả Charles Petzold.
- Charles Petzold, Programming Windows, 1998.
- Lê Hữu Đạt, Lập trình Windows, Nhà xuất bản giáo dục, 2000.
- Gs. Phạm Văn Ất, Lập trình Windows, Nhà xuất bản Giao thông Vận tải, 2005.



Tài liệu tham khảo

- Leonard Richbardson & Mike Amundsen, Restful Web APIs, O'REILLY
- https://www.restapitutorial.com/
- https://www.restapitutorial.com/resources.html
- https://www.vinaysahni.com/best-practices-for-a-pragmatic-restful-api

Tài liệu tham khảo

- https://developers.facebook.com/docs/
- https://giaiphapseo.com/facebook-la-gi/
- https://viblo.asia/p/facebook-graph-api-maGK7mPMlj2
- https://developers.facebook.com/tools/explorer/
- https://developers.facebook.com/docs/graph-api/explorer
- https://www.youtube.com/watch?v=eVlNJ1X_YXQ&list=RDCMUCl85rcoLszFv_qCL1hcOB2g&start_radio=1&t=146

Các thành phần của máy tính hiện đại

- Bộ vi xử lý
- Mainboard
- Nguồn điện
- Card màn hình
- Màn hình
- RAM
- Ổ cứng
- Bàn phím
- Chuột
- Ổ CD/DVD

Chỉ có phần cứng (hardware) là chưa đủ!

- Máy tính cần có phần mềm (software) hỗ trợ:
  - Phần mềm hệ thống
    - Điều khiển công việc của máy tính và các thiết bị ngoại vi
    - Cung cấp môi trường thực hiện cho các chương trình ứng dụng
  - Chương trình ứng dụng: giải quyết những bài toán thực tế của người sử dụng
  - Các phần mềm ứng dụng có thể là ứng dụng windows hoặc ứng dụng Web.

Hệ điều hành

- Là thể loại chính của phần mềm hệ thống
- Là tập hợp các chương trình để hỗ trợ:
  - Điều khiển các phần cứng của máy tính,
  - Làm việc với các tệp tin,
  - Nhập/xuất dữ liệu,
  - Thực hiện các chương trình ứng dụng và các tiện ích

Hệ điều hành (HĐH)

- HĐH cung cấp một tập hợp các hàm hệ thống (system calls) để điều khiển tài nguyên của máy tính một cách thuận tiện:
  - Về nguyên tắc, việc thực hiện các lời gọi hệ thống được làm việc trong chế độ đặc quyền của bộ vi xử lý
  - Các chương trình của người sử dụng làm việc trong chế độ không đặc quyền

API là gì?

- **API - Application Programming Interface** (Giao diện lập trình ứng dụng)
  - Không phải là ngôn ngữ lập trình
  - Là một giao tiếp phần mềm được dùng bởi các ứng dụng khác nhau.

Windows API là gì?

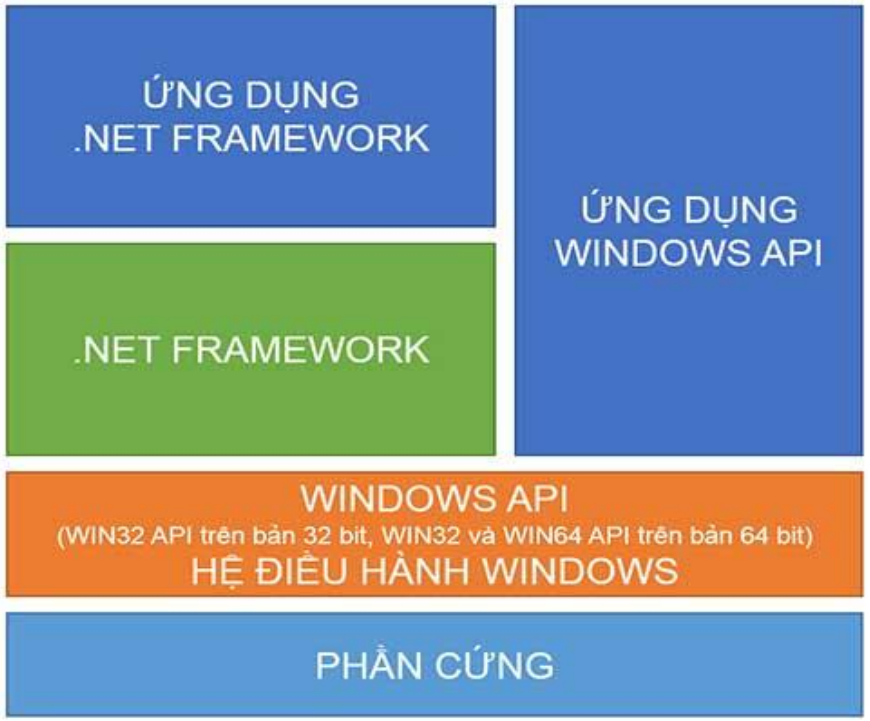
- Windows cung cấp các hàm để người lập trình có thể thâm nhập vào các đặc trưng của hệ điều hành, các hàm này gọi là các API của windows.
- Những hàm này được đặt trong thư viện liên kết động (Dynamic Link Library – DLL). Các chương trình ứng dụng sử dụng chúng thông qua các lời gọi hàm và chỉ chia sẻ được khi trong máy tính có cài đặt Windows.

Windows API là gì?

- Mỗi hệ điều hành đều có những bộ API khác nhau. Nó cung cấp cho người lập trình các hàm tương tác với Cơ sở dữ liệu, máy in, hệ đồ họa…
  - Ví dụ: Các hàm API của Windows cho phép chúng ta lập trình ứng dụng thay đổi icon của cửa sổ, xem thông tin máy tính, kết nối với máy in…
  - Hay các hàm API của Google cho phép lập trình viên lấy thông tin, sửa thông tin người dùng…
- Ngày nay các HĐH Windows có tới hàng ngàn hàm API

Windows API là gì?

- Windows API là giao diện lập trình nằm ngay trên nền Windows, cung cấp các hàm thao tác trực tiếp với hệ điều hành và phần cứng máy tính. Các ứng dụng Windows sẽ thông qua Windows API để thao tác với máy tính.



Mô hình Windows API

Web API là gì?

- API dưới dạng web.
- Nó thường là hệ thống gồm: các thông điệp dạng HTTP, NET/TCP,.. để gửi lên web, sau đó nhận các thông điệp phản hồi về (thường là dạng JSON, XML có cấu trúc rõ ràng).
- Nó cung cấp cho người dùng một tập các hàm giúp cho người dùng có thể tương tác được với các dịch vụ được cung cấp trên web. Người dùng thông qua các hàm (API) để sử dụng các dịnh vụ đó.
- VD: các hàm, các giao thức, gói tin gửi đi, các URL,..

Web API là gì?

- **Lợi ích** của việc sử dụng web API là:
  - giải quyết vấn đề về dung lượng lưu trữ, tốc độ xử lý….;
  - Cập nhật realtime (thời gian thực);
  - Tăng tính tiện dụng;
  - Giảm thời gian, chi phí khi phát triển ứng dụng; ….

Các dạng Web API

- **Địa chỉ web của API**: Mỗi API sẽ có địa chỉ web tương ứng, truy vấn các địa chỉ này và các tham số truyền vào để sử dụng.
  - VD: Google map: **http://maps.googleapis.com/maps/api/**
  - Google Translator: **http://translate.googleapis.com/translate_static**
  - Yahoo! Map Service: **http://local.yahooapis.com/MapsService/**

Các dạng Web API

- **Các thư viện động:** Web API được đóng gói trong 1 bộ thư viện. Khi sử dụng, gọi các hàm trong đó để sử dung, tương tác với dịch vụ ở trên Net. VD: Facebook SDK, Youtube Data API..
- **Các thư viện Javascript:** Dùng cho việc sử dụng Web API nhúng vào trong website của người dùng (có dạng gần giống với cách sử dụng địa chỉ web API). VD: Google, Youtube, Youtube Player API,..

Môi trường lập trình

- Sử dụng môi trường Visual sudio phiên bản 2015 trở lên.

Chương trình đơn giản nhất cho Windows



```
#include <windows.h>
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, PSTR szCmdLine, int iCmdShow)
{

    return 0;
}
```

Hàm WinMain()

- Hàm WinMain là điểm vào của các ứng dụng trong Windows
- Các nhiệm vụ của hàm WinMain:
  - Khởi tạo ứng dụng
  - Hiển thị cửa sổ chính của ứng dụng
  - Tạo vòng lặp lấy ra từng thông điệp rồi xử lý
  - Trả ra một giá trị kiểu nguyên cho HĐH

Các tham số của hàm WinMain

- **hInstance, hPrevInstance**: Chỉ số chương trình khi chúng đang chạy. hInstance là chỉ số bản chương trình vừa khởi động, hPrevInstance là chỉ số bản chương trình khởi động trước nó (thường là Null)
- **szCmdLine**: Là dòng lệnh sử dụng để chạy chương trình.
- **iCmdShow**: cho biết ban đầu chương trình được hiển thị như thế nào trên màn hình – có thể là hiển thị bình thường hoặc là hiển thị tối đa để lấp đầy màn hình hoặc được thu nhỏ để hiện thị trên thanh danh sách nhiệm vụ (task list bar).

➔ Các tham số trên do HĐH truyền vào

Ứng dụng HelloWorld


```
#include <windows.h>
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, PSTR szCmdLine, int iCmdShow)
{
    MessageBox(NULL,TEXT("Hello World") ,TEXT("Hi"), MB_OK | MB_ICONINFORMATION);
    return 0;
}
```



Hàm MessageBox

- **Tham số đầu**: handle (chỉ số định danh) của cửa sổ
- **Tham số thứ 2**: Là Nội dung hộp thông báo.
- **Tham số thứ 3**: Là Tiêu đề hộp thông báo.
- **Tham số thứ 4**: Sự kết hợp của kiểu chức năng và biểu tượng. Bắt đầu bằng tiền tố MB_

Hàm MessageBox


```
#define MB_OK                       0x00000000L
#define MB_OKCANCEL                 0x00000001L
#define MB_ABORTRETRYIGNORE         0x00000002L
#define MB_YESNOCANCEL              0x00000003L
#define MB_YESNO                    0x00000004L
#define MB_RETRYCANCEL              0x00000005L
```

Khi bạn chọn đối số thứ 4 có giá trị là 0 trong tệp HelloMsg.c thì chỉ có nút OK xuất hiện. Bạn có thể sử dụng toán tử OR (|) để tổ hợp một trong những hằng ở trên với một hằng mà nó cho biết nút nào được chọn là mặc định:

```
#define MB_DEFBUTTON1               0x00000000L
#define MB_DEFBUTTON2               0x00000100L
#define MB_DEFBUTTON3               0x00000200L
#define MB_DEFBUTTON4               0x00000300L
```

Bạn cũng có thể sử dụng một hằng mà nó cho biết hình dáng của một icon trong hộp thông báo:

```
#define MB_ICONHAND                 0x00000010L
#define MB_ICONQUESTION             0x00000020L
#define MB_ICONEXCLAMATION          0x00000030L
#define MB_ICONASTERISK             0x00000040L
```

Một trong những icon này có cái tên khác:

```
#define MB_ICONWARNING              MB_ICONEXCLAMATION
#define MB_ICONERROR                MB_ICONHAND
#define MB_ICONINFORMATION          MB_ICONASTERISK
#define MB_ICONSTOP                 MB_ICONHAND
```

Hàm MessageBox

- Hàm MessageBox trả về giá trị mà người dùng chọn (giá trị này có kiểu int). Các giá trị có thể là:
  - IDYES
  - IDNO
  - IDCANCEL
  - IDABORT
  - …

Kiến trúc Windows

- Lập trình trong Windows là lập trình hướng đối tượng – các cửa sổ
- Các dạng cửa sổ:
  - Cửa sổ ứng dụng
  - Hộp hội thoại
  - Các nút nhấn, nút radio, hộp kiểm tra, hộp danh sách, thanh cuốn, hộp soạn thảo
- Cửa sổ nhận các dữ liệu đầu vào từ người sử dụng theo hình thức "các thông điệp" tới cửa sổ đó
- Cửa sổ cũng có thể sử dụng các thông điệp để liên lạc với các cửa sổ khác

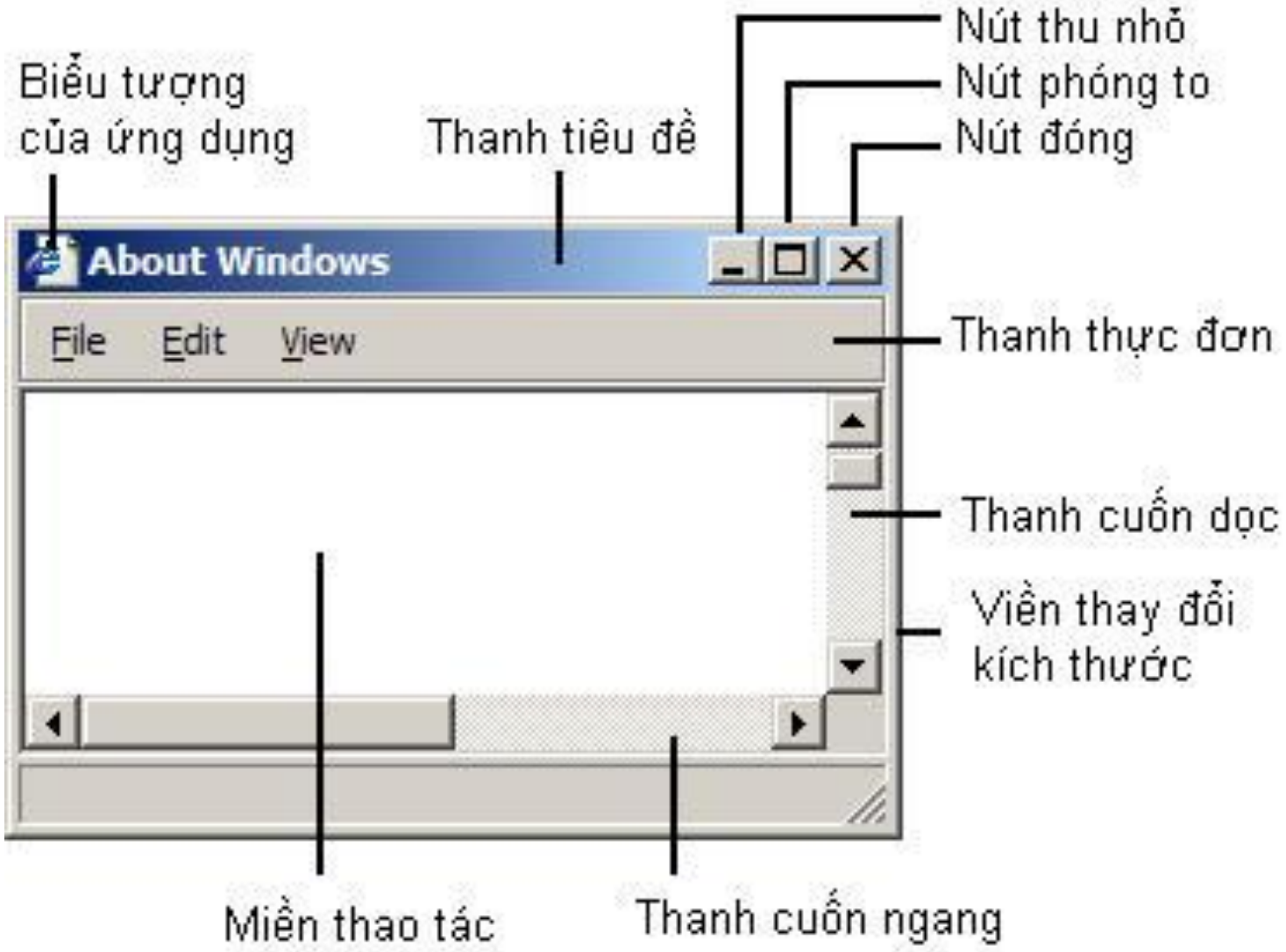
Cửa sổ

- Lập trình windows là lập trình hướng đối tượng. Mỗi đối tượng trong Windows có một cái tên, mỗi đối tượng mang những nét đặc trưng riêng, và mỗi đối tượng được biết đến như một "cửa sổ".
- Cửa sổ – là một vùng hình chữ nhật trên màn hình để:
  - Hiển thị các thông tin xuất
  - Nhận các thông tin nhập từ người sử dụng
- Tại một thời điểm chỉ có 1 cửa sổ có thể nhận thông tin nhập từ người sử dụng!!!

Cửa sổ chính của ứng dụng

- Là cửa sổ cha của các cửa sổ khác được tạo ra trong ứng dụng

Các thành phần của cửa sổ



Miền thao tác/miền phi thao tác

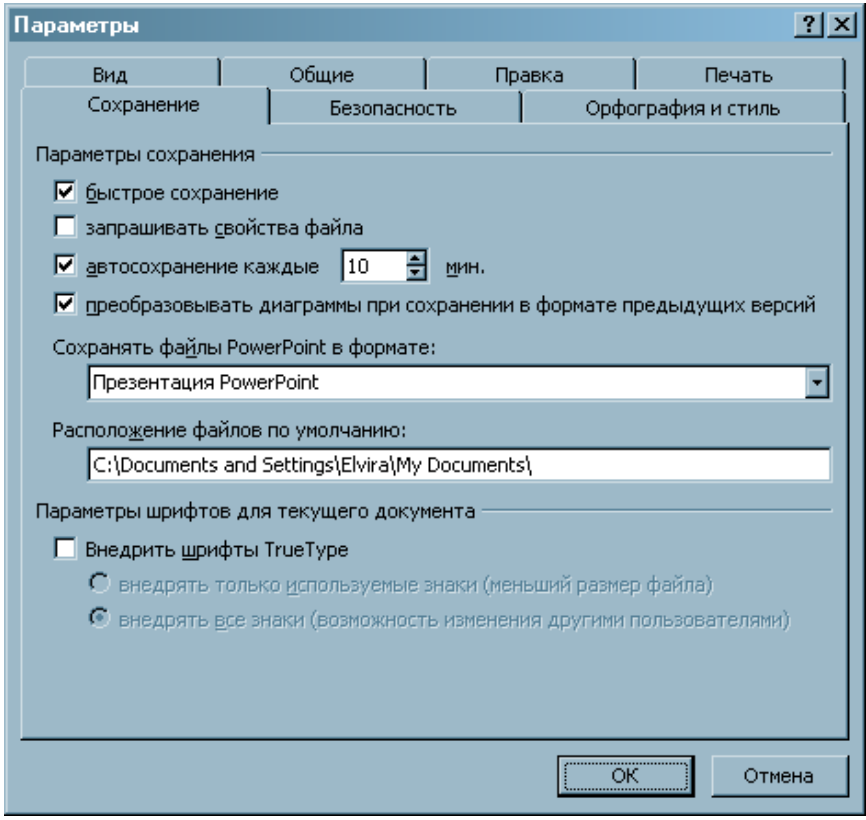
- Một cửa sổ gồm 2 phần
  - Miền thao tác
  - Miền phi thao tác
- Miền thao tác
  - Dùng để hiển thị văn bản và đồ hoạ
  - Được điều khiển bởi ứng dụng
- Miền phi thao tác: Về nguyên tắc, điều khiển miền phi thao tác là công việc của HĐH

Các điều khiển (control)

- Là các cửa sổ con dùng để nhập, xuất thông tin
  - Hộp soạn thảo (Edit box)
  - Văn bản tĩnh (Static Text)
  - Nút (Button)
  - Hộp danh sách (List box)
  - ….

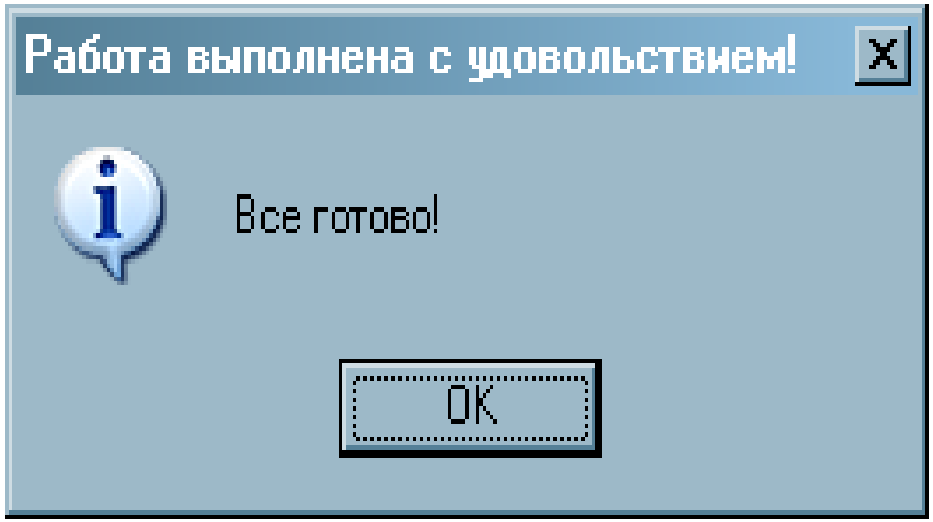
Các hộp hội thoại

- Là cửa sổ mà chứa 1 hay nhiều phần tử điều khiển
- Thông thường các hộp hội thoại không chứa:
  - Thực đơn
  - Các thanh cuốn
  - Các nút thu nhỏ/phóng to



Các cửa sổ thông báo (Message box)

- Là các cửa sổ hội thoại đặc biệt dùng để hiển thị các thông báo ngắn tới người sử dụng:
  - Khuyến cáo về các lỗi
  - Thông báo về việc hoàn thành các thao tác



Thông điệp

- Một cửa sổ nhận các dữ liệu đầu vào từ người sử dụng theo hình thức "các thông điệp" tới cửa sổ đó.
- Một cửa sổ cũng có thể sử dụng các thông điệp để liên lạc với các cửa sổ khác.
- Việc làm quen với khái niệm "thông điệp" là một phần quan trọng trong việc học viết các chương trình cho Windows.

Quy trình hoạt động của chương trình ứng dụng

- Cửa sổ được hiển thị lên màn hình.
- Windows chờ cửa sổ gửi thông điệp.
- Các thông điệp được Windows gửi trả lại chương trình ứng dụng thông qua lời gọi hàm của chúng trong chương trình ứng dụng.
- Khi nhận được thông điệp, chương trình ứng dụng gọi các hàm API và hàm của riêng chúng để thực hiện công việc mong muốn.

Tạo cửa sổ

- Một cửa sổ phải được tạo từ một lớp cửa sổ đã được đăng ký trong HĐH
- Khi tạo cửa sổ, ứng dụng phải chỉ ra những đặc tính sau của cửa sổ:
  - Tên lớp cửa sổ
  - Tên cửa sổ
  - Kiểu cách cửa sổ
  - Vị trí và kích thước cửa sổ
  - Handle của cửa sổ cha
  - Handle của thực đơn
  - Handle của ứng dụng

Lớp cửa sổ

- Là một tập hợp xác định gồm các đặc tính được sử dụng để tạo khuôn mẫu cho các cửa sổ
  - Mỗi cửa sổ đều thuộc vào một lớp cửa sổ xác định
  - Từ một lớp cửa sổ, ứng dụng có thể tạo một số lượng tuỳ ý các cửa sổ
  - Các cửa sổ thuộc vào cùng một lớp cửa sổ sẽ có những tính chất, hành vi giống nhau

Các lớp cửa sổ có sẵn

- Trong hệ thống đã đăng ký một số lớp cửa sổ có sẵn
  - BUTTON
  - LISTBOX
  - COMBOBOX
  - STATIC
  - EDIT
  - MDICLIENT
  - RICHEDIT_CLASS
  - SCROLLBAR

Tên cửa sổ

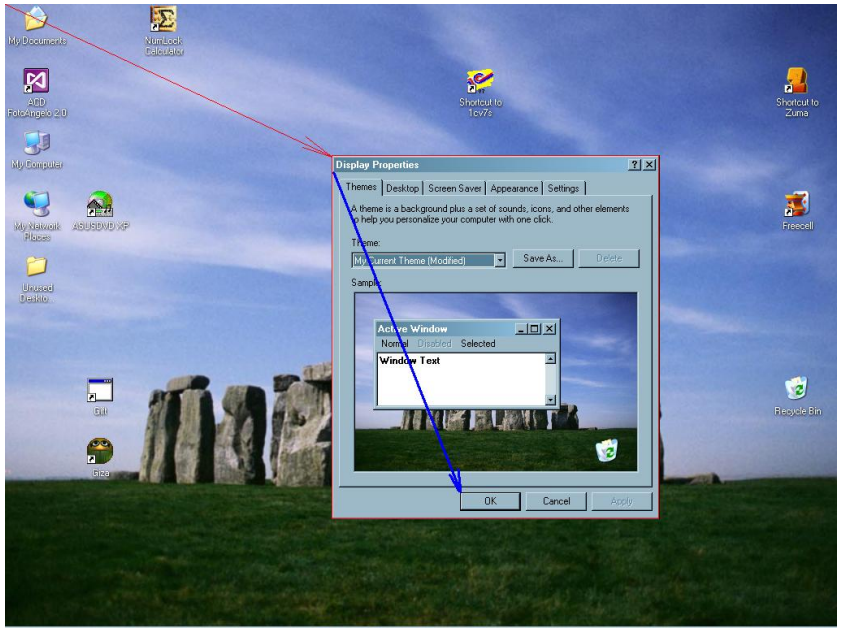
- Tên của cửa sổ được hiển thị ở trên dòng tiêu đề đối với
  - Cửa sổ chính của ứng dụng
  - Các hộp hội thoại
- Đối với một số phần tử điều khiển – tên được hiển thị bên trong miền thao tác
  - Edit box
  - Static text
  - Button
- Đối với một số phần tử điều khiển – tên hoàn toàn không được hiển thị
  - List box
  - Combo box

Kiểu cách cửa sổ

- Kiểu cách cửa sổ – định nghĩa các khía cạnh xác định của hành vi và hình dáng bề ngoài của cửa sổ
- Một cửa sổ có thể có nhiều kiểu cửa sổ
- Một số kiểu có thể được áp dụng cho tất cả các cửa sổ, trong khi đó một số khác – chỉ áp dụng cho các cửa sổ thuộc lớp xác định

Vị trí và kích thước cửa sổ

- Vị trí và kích thước của cửa sổ được tính theo đơn vị điểm ảnh
- Vị trí của cửa sổ – là toạ độ của góc trên bên trái của cửa sổ so với
  - Góc trên bên trái của màn hình
  - Hoặc so với cửa sổ cha, nếu đó là cửa sổ con



Quan hệ cha-con

- Một số cửa sổ có thể có cha
  - Những cửa sổ đó được gọi là cửa sổ con
  - Các cửa sổ cha xác định hệ thống toạ độ cho các cửa sổ con của mình
- Các cửa sổ không có cha – là các cửa sổ ở mức đỉnh

Handle của ứng dụng

- Được truyền vào qua tham số của hàm WinMain
- Được sử dụng khi tạo cửa sổ và làm việc với các tài nguyên, các luồng, …

Handle của cửa sổ

- Mỗi cửa sổ sau khi được tạo đều gắn với 1 handle (chỉ số) riêng biệt
- Dùng để xác định cửa sổ một cách thống nhất trong hệ thống
- Có kiểu HWND
- Ứng dụng có thể truyền handle vào trong các hàm khác nhau để làm việc với cửa sổ tương ứng

Định nghĩa lớp cửa sổ



- Sử dụng cấu trúc WNDCLASS để lưu các thông số lớp cửa sổ
- Gán các giá trị ban đầu cho lớp cửa sổ

```
WNDCLASS     wndclass; //Khai báo lớp cửa sổ
static TCHAR szAppName[] = TEXT("Ví dụ 1"); //Tên ứng dụng

wndclass.style = CS_HREDRAW | CS_VREDRAW; //Kiểu cửa sổ
wndclass.lpfnWndProc = WndProc; //Thủ tục cửa sổ (thủ tục sử lý các thông điệp
wndclass.cbClsExtra = 0; //Không cần thông tin thêm cho cửa sổ
wndclass.cbWndExtra = 0;
wndclass.hInstance = hInstance; //Chỉ số cửa cửa sổ
wndclass.hIcon = LoadIcon(hInstance, szAppName); //Tải ICON cho cửa sổ
wndclass.hCursor = LoadCursor(NULL, IDC_ARROW); //Tải chuột
wndclass.hbrBackground = (HBRUSH)GetStockObject(WHITE_BRUSH); //màu nền của cửa sổ
wndclass.lpszMenuName = NULL; //menu
wndclass.lpszClassName = szAppName; //Tên lớp ứng dụng, szAppName có kiểu dữ liệu TCHAR được khai báo là static
```

Đăng ký lớp cửa sổ

- Ứng dụng cần phải đăng ký lớp cửa sổ cho cửa sổ chính của mình
- Đăng ký lớp cửa sổ được thực hiện nhờ hàm RegisterClass()

```
if (!RegisterClass(&wndclass))
{
    MessageBox(NULL, TEXT("This program requires Windows NT!"),
        szAppName, MB_ICONERROR);
    return 0;
}
```

Tạo cửa sổ

- Dùng hàm CreateWindow()
- Sau khi được tạo, cửa sổ còn ở dạng ẩn!!!
- Để hiển thị cửa sổ - gọi hàm ShowWindow()
- Ví dụ:

```
hwnd = CreateWindow(szAppName,  //Tên lớp cửa sổ
    TEXT("About Box Demo Program"), //Tiêu đề
    WS_OVERLAPPEDWINDOW, //Kiểu cửa sổ
    CW_USEDEFAULT, CW_USEDEFAULT, //Tọa độ trên bên trái của cửa sổ
    CW_USEDEFAULT, CW_USEDEFAULT, //Kích thước của cửa sổ
    NULL, NULL, hInstance, NULL); //chỉ số của cửa sổ cha, menu, chỉ số cửa sổ hiện tại,
                                  //con trỏ đến thông tin phụ khác

ShowWindow(hwnd, iCmdShow);
UpdateWindow(hwnd);
```

Thông điệp

- Xử lý các thông điệp là công việc của ứng dụng trong Windows
- Các thông điệp có thể được tạo bởi
  - Hệ thống
  - Ứng dụng
- Thông điệp được tạo mỗi khi xảy ra 1 sự kiện trong Windows:
  - Di chuyển chuột
  - Nhấn phím
  - Các sự kiện của timer, …

Cấu trúc thông điệp


```
typedef struct  tagMSG
{
    HWND hwnd;              // Handle của cửa sổ
    UINT message;           // Định danh của thông điệp
    WPARAM wParam;          // Tham số wParam
    LPARAM lParam;          // Tham số lParam
    DWORD time;             // Thời gian gửi thông điệp
    POINT pt;               // Vị trí chuột
} MSG;
```

Hàng đợi thông điệp

- Hệ thống chuyển thông điệp vào hàng đợi thông điệp của luồng tương ứng (thread)
  - Thread – đó là một luồng thực hiện độc lập của chương trình bên trong 1 tiến trình (process)
  - Bên trong tiến trình tồn tại ít nhất 1 luồng

Vòng lặp thông điệp

- Lấy từng thông điệp ra từ hàng đợi rồi xử lý
- Thông thường vòng lặp thông điệp có dạng sau:

```
MSG msg;
while (GetMessage(&msg, NULL, 0, 0))
{
    TranslateMessage(&msg);
    DispatchMessage(&msg);
}
```

Vòng lặp thông điệp

- **GetMessage()** - lấy ra 1 thông điệp từ hàng đợi
- **TranslateMessage()**
  - Chuyển 1 thông điệp phím ảo sang thông điệp ký tự (nếu có thể)
  - Thông điệp ký tự này sẽ được đưa vào hàng đợi thông điệp và được lấy ra ở lần lấy tiếp theo
- **DispatchMessage()** – truyền thông điệp vào thủ tục cửa sổ tương ứng để xử lý

Thông điệp WM_CREATE

- Báo hiệu rằng, cửa sổ đang được tạo.
- Nó được tạo trong quá trình thực hiện hàm CreateWindow()
- Thủ tục cửa sổ của cửa sổ mới sẽ nhận được thông điệp này sau khi cửa sổ được tạo nhưng trước khi nó được hiển thị

Thông điệp WM_SIZE, WM_SIZING

- WM_SIZE được gửi tới cửa sổ sau khi kích thước của nó bị thay đổi
- WM_SIZING được gửi tới cửa sổ trong khi kích thước của nó đang bị thay đổi

Thông điệp WM_PAINT

- Được gửi tới cửa sổ trong trường hợp một miền của cửa sổ cần phải được vẽ lại, nó xảy ra khi:
  - Ứng dụng gọi hàm RedrawWindow() hoặc UpdateWindow()
  - Một miền của cửa sổ trở nên vô hiệu lực khi nó được kích hoạt lên mức đỉnh

Thông điệp WM_DESTROY

- Được gửi tới cửa sổ sau khi nó được dọn khỏi màn hình và trước khi nó bị tiêu huỷ
- Tiếp theo các cửa sổ con của nó cũng nhận được thông điệp này
- Thông thường ứng dụng xử lý thông điệp này bằng cách gọi hàm PostQuitMessage

Thông điệp WM_CLOSE

- Báo hiệu rằng một cửa sổ hoặc một ứng dụng cần được kết thúc
- Ứng dụng có thể yêu cầu người sử dụng xác nhận trước khi hủy cửa sổ bằng cách:
  - Xử lý WM_CLOSE
  - Và chỉ gọi hàm DestroyWindow nếu người sử dụng chấp nhận hủy



Gửi thông điệp WM_DESTROY tới cửa sổ

Thông điệp WM_QUIT

- Thông điệp này được gửi tới ứng dụng (chứ không phải cửa sổ), nó biểu thị yêu cầu kết thúc ứng dụng
- Hàm GetMessage() trả ra giá trị 0 (false) khi lấy ra thông điệp WM_QUIT ⟹ Kết thúc vòng lặp thông điệp ⟹ Kết thúc hàm WinMain ⟹ Kết thúc ứng dụng

Thủ tục cửa sổ

- Mỗi cửa sổ có 1 thủ tục cửa sổ gắn với nó
- Tất cả các cửa sổ thuộc cùng 1 lớp cửa sổ sẽ sử dụng cùng 1 thủ tục cửa sổ
- Hành vi của cửa sổ được xác định bởi phản ứng của thủ tục cửa sổ đối với các thông điệp nhận được

Đăng ký thủ tục cửa sổ

- Đăng ký thủ tục cửa sổ được thực hiện trong quá trình đăng ký lớp cửa sổ
- Ứng dụng có thể tạo một số lượng tuỳ ý các cửa sổ thuộc cùng một lớp cửa sổ và sử dụng cùng một thủ tục cửa sổ

Nguyên mẫu của thủ tục cửa sổ


```
LRESULT CALLBACK WindowProc(
    HWND hwnd,
    UINT uMsg,
    WPARAM wParam,
    LPARAM lParam);
```

Thủ tục cửa sổ mặc định

- Đối với nhiều thông điệp, trong hệ thống có định nghĩa sẵn hành vi mặc định, ví dụ như:
  - Di chuyển cửa sổ
  - Thu nhỏ/phóng to
  - Phản ứng với di chuyển của chuột
- Tất cả các thông điệp mà không được xử lý rõ ràng trong thủ tục cửa sổ của ứng dụng, thì chúng phải được chuyển tới thủ tục cửa sổ mặc định bằng cách gọi hàm DefWindowProc()

Ví dụ đơn giản nhất về thủ tục cửa sổ



```
LRESULT CALLBACK MainWndProc(
    HWND hWnd,
    UINT uMsg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch(uMsg)
    {
    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProc(hWnd, uMsg, wParam, lParam);
}
```



Các bước xây dựng ứng dụng cho Windows

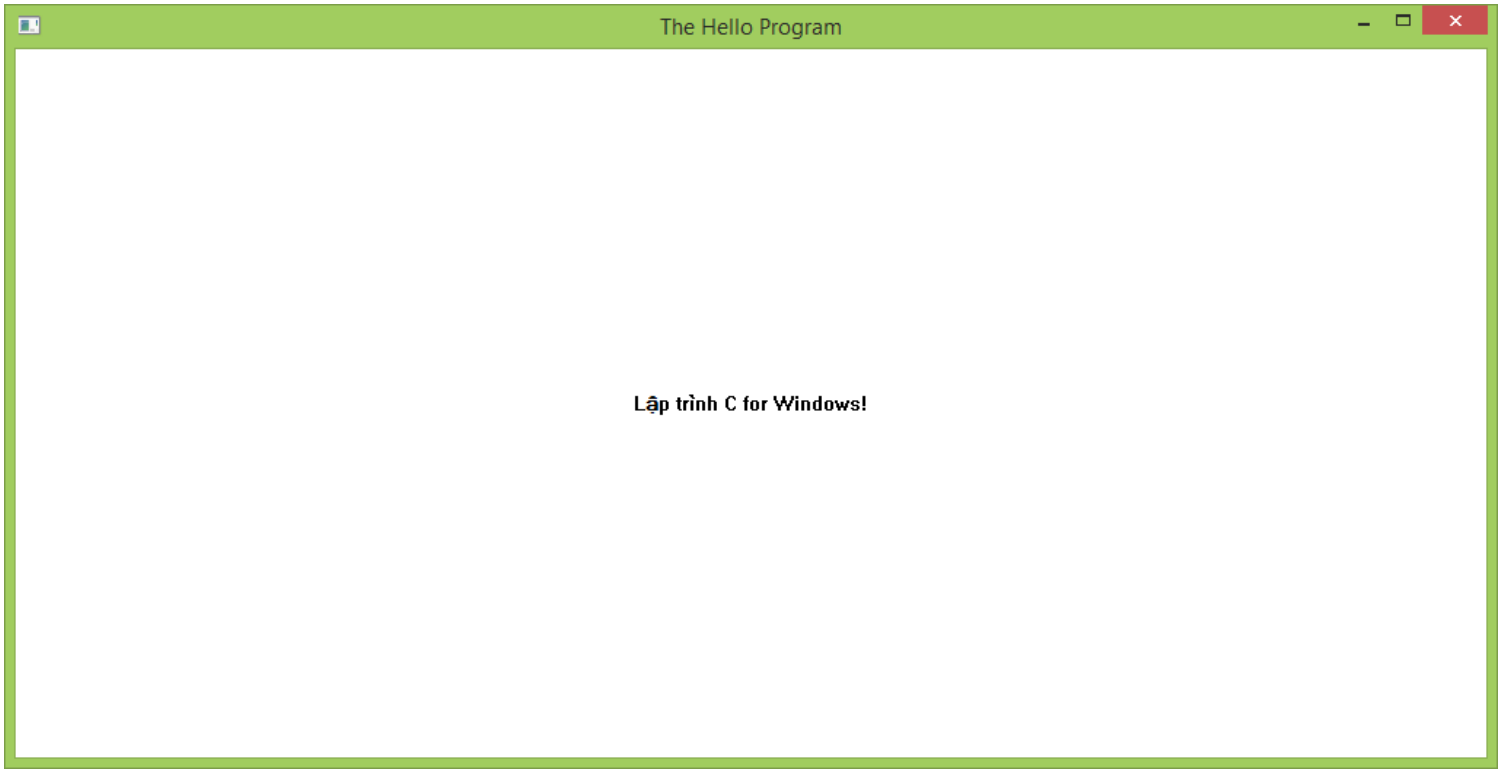
- Định nghĩa và đăng ký lớp cửa sổ
- Tạo cửa sổ chính của ứng dụng
- Tạo vòng lặp thông điệp
- Định nghĩa thủ tục cửa sổ - định nghĩa cách thức xử lý từng loại thông điệp gửi tới cửa sổ

** 3 bước đầu tiên được viết trong hàm WinMain



Chương trình Đầu tiên

- Hiển thị "Lập trình C for windows" ở trung tâm của miền thao tác



Phân tích chương trình

- Chương trình chứa một lượng mã nguồn khá nhỏ nhưng lại có khá nhiều chức năng:
  - Ta có thể di chuyển cửa sổ xung quanh màn hình
  - Thay đổi kích thước cửa sổ - vị trí của dòng văn bản cũng thay đổi tương ứng
  - Có các nút Maximize/Minimize, Close, thực đơn hệ thống

Các hàm API của Windows

- LoadIcon - Tải một biểu tượng
- LoadCursor - Tải một con trỏ chuột
- GetStockObject - Lấy một đối tượng đồ hoạ, trong trường hợp này là chổi quét để sử dụng cho việc vẽ nền cửa sổ
- RegisterClass - Đăng ký một lớp cửa sổ
- MessageBox - Hiển thị một hộp thông báo
- CreateWindow - Tạo một cửa sổ từ lớp cửa sổ đã đăng ký

Các hàm API của Windows

- ShowWindow - Hiển thị cửa sổ
- UpdateWindow - Khiến cửa sổ vẽ lại chính nó
- GetMessage - Lấy ra một thông điệp từ hàng đợi thông điệp
- TranslateMessage - Chuyển đổi một số thông điệp bàn phím
- DispatchMessage - Gửi một thông điệp đến thủ tục cửa sổ đích

Các hàm API của Windows

- BeginPaint - Khởi đầu cho công việc vẽ nội dung cửa sổ
- EndPaint - Kết thúc công việc vẽ nội dung cửa sổ
- GetClientRect - Lấy kích thước của miền thao tác
- DrawText - Hiển thị một xâu ký tự
- PostQuitMessage - Đưa thông điệp kết thúc chương trình vào hàng đợi thông điệp
- DefWindowProc - Thực hiện công việc xử lý mặc định (của Windows) đối với một thông điệp

Các kiểu dữ liệu mới

Stt	Kiểu dữ liệu	Chú thích
1	HANDLE	(*nguyên không dấu 16 bit*) dùng để định danh đối tượng • HWND : window • HMENU : menu • HCURSOR : cursor
2	HBRUSH	(*brush*) mẫu tô: solid, dash, dot, cross, …
3	HPALLETE	(*pallete*) bảng màu
4	HFONT	(*font*) Facename, size, style
5	HBITMAP	*bitmap*
6	HICON	*icon*
7	HPEN	Nét vẽ: solid, dot, dash, size, color
8	HINSTANCE	Instance
9	HDC	Device context
10	LTSTR	(*long pointer string*) con trỏ đến chuỗi ký tự
11	WPARAM LPARAM	(*word*) các tham số đi kèm message.
12	LRESULT	(*long*) kiểu trả về của hàm xử lý Message.
13	LPVOID	Con trỏ đến kiểu dữ liệu bất kỳ.

Các tiền tố

Tiền tố	Hằng
CS	Kiểu lớp
CW	Tham số cho việc tạo cửa sổ
DT	Tham số cho việc vẽ văn bản
IDI	Định danh của biểu tượng
IDC	Định danh của con trỏ chuột
MB	Tham số cho hộp thông báo
SND	Tham số cho âm thanh
WM	Thông điệp
WS	Kiểu cửa sổ

Bài tập về nhà

- Thay đổi chương trình ví dụ trên với yêu cầu:
  - Khi người dung click chọn nút Close của cửa sổ sẽ hiển thị thông báo: "Bạn có muốn thoát không?" với 2 nút Yes/No. Nếu chọn Yes thì thoát, no thì không. → **Viết thông điệp WM_CLOSE.**
  - Thay đổi kích thước, con trỏ chuột, tiêu đề của cửa sổ.